ESPRO POS SYSTEM

UNIFIED RETAIL, RECIPE & INVENTORY ARCHITECTURE

CAPSTONE TECHNICAL SPECIFICATION

Data Warehousing & ETL Pipeline Architecture

End-to-End Operational to Analytical Data Processing Pipeline

Comprehensive architectural design and engineering specification detailing the flow of coffee shop operations—from high-velocity Point-of-Sale checkouts, recipe explosions, batch FEFO allocations, and employee time-clock attendance to staging, atomic transformation rules, data quality validation ("Stop & Alert"), multi-dimensional Star Schema storage, and executive OLAP reporting views.

LAYER 01

Operational Sources

POS sales transactions, recipe ingredients, FEFO batches, expenses, employee attendance, and offline localStorage cache.

LAYER 02

ETL Processing Engine

Unit conversion, recipe explosion, COGS snapshots, packaging rules, FEFO sorting, and Stop & Alert quality checks.

LAYER 03

Star Schema Warehouse

Fact_Sales, Fact_Inventory_Movement, Fact_Expenses, Fact_Attendance, 10 Dimensions, and 4 Localized Data Marts.

LAYER 04

OLAP Analytical Views

Multi-dimensional slice, dice, roll-up, drill-down engine serving Owner P&L, Store Manager Inventory, and Cashier Views.

COURSE / DEGREE
BS Information Technology / CS Capstone Project

PROJECT TITLE
Espro POS: Coffee Shop Management & BI System

ARCHITECTURE VERSION
v6.0 Enterprise Pipeline Specification

DATE
October 2026

BRAND PALETTE:

End-to-End Data Warehousing & ETL Pipeline Flowchart

FULL SYSTEM ARCHITECTURE: OPERATIONAL INGESTION → ETL PROCESSING → STAR SCHEMA → OLAP ANALYTICS

FIGURE 1.0 ▪ COMPREHENSIVE ARCHITECTURE

LAYER 1: DATA SOURCES — OLTP

POS Sales Transactions — SALES

Checkout sales data: items JSONB, subtotal, discount, total, payment_method (cash/gcash), order_type, cashier_name, status.

Inventory & Batches — STOCK

Raw ingredients, stock_quantity, low_stock_threshold, cost_per_unit, actual_purchase_price. ingredient_batches (FEFO expiry) & stock_movements ledger.

Menu & Recipe Master — MASTER

menu_items, sizes (ingredient_multiplier), recipes (menu_item_ingredients), addons, addon_ingredients, order_supply_rules (packaging).

Operating Expenses — EXPENSES

Daily store expenses: Utilities, Supplies, Salaries, Rent, Maintenance, Marketing, Others with amount, date, and description.

HR & Attendance Kiosk — HR

employees (code, PIN, rate), shift_templates, shift_schedules, attendance_records (clock_in/out, hours, overtime, tardiness).

Offline Browser Cache — STORAGE

localStorage resilience fallback when network fails; profiles & Supabase Auth user accounts.

LAYER 2: ETL PIPELINE — TRANSFORM

Extract & Staging Area — DB.TS

loadSales, loadIngredients, loadBatches, loadExpenses. PostgREST fetch + localStorage fallback; in-memory React state staging.

TRANSFORMATION RULES (ORDERCALC & UNITCONVERSION)

1. **Unit Conversion:** Normalize g/kg/ml/L/shots/pumps/pcs
2. **Recipe Explosion:** computeLine size multiplier & addon chains
3. **COGS Snapshot:** Split ingredients COGS vs supplies COGS
4. **Packaging Rules:** computeOrder takeout packaging supplies
5. **Net Metric:** Net Sales = Subtotal (Gross) - Discount
6. **Missing Fill:** Legacy cost fallback; default dine_in
7. **FEFO Allocation:** Earliest-expiry-first; expire_batches
8. **Deduplication:** Deterministic cart merging via lineKey
9. **Date Normalization:** Asia/Manila (UTC+8) day boundaries

◆ QUALITY CHECK & VALIDATION

- Stock Shortage Check (shortages.length == 0)
- Compatible Units & Positive Qty/Amounts
- Discount Clamping (discount ≤ subtotal)
- PIN & Void Status Integrity Verification

⚠ STOP & ALERT ROUTINE (FAIL)

- Rollback Atomic RPC (PL/pgSQL Exception)
- Toast Error Alert (react-hot-toast)
- TopBar Low-Stock & Near-Expiry Alert Badge

✓ LOAD PHASE: ATOMIC RPCS (PASS)

complete_sale_with_deduction, void_sale_with_restoration, restock_ingredient, adjust_ingredient_stock.

LAYER 3: DATA WAREHOUSE STAR SCHEMA

Dimensions (Context) — 10 DIMS

- Dim_Date (date_key, month, week, year)
- Dim_Product (menu_items.id, category)
- Dim_Size & Dim_Addon
- Dim_Ingredient_Supply (unit, cost)
- Dim_Employee_Cashier (code, role)
- Dim_OrderType & Dim_PaymentMethod
- Dim_ExpenseCategory & Dim_Batch

Fact_Sales (Measurements) — CORE FACT

Grain: Sale Line Item
Metrics: Quantity, Unit Price, Gross Amount, Discount Amount, *Net Amount = Gross - Discount*, COGS Ingredients, COGS Supplies, Net Profit.

Satellite Fact Tables — FACTS

- Fact_Inventory_Movement (stock_movements)
- Fact_Expenses (daily operational expenses)
- Fact_Attendance (hours, overtime, late_mins)

Localized Data Marts — MARTS

- **Sales Mart:** Daily/weekly/monthly revenue & tickets
- **Inventory Mart:** Stock valuation, spoilage, turnover
- **Finance Mart:** P&L, Gross Margin, OPEX, Net Profit
- **Workforce Mart:** Labor hours, overtime expenditure

LAYER 4: OLAP & VIEWS — ANALYTICS

OLAP Analytical Engine — USEMEMO

Multi-dimensional in-memory engine (Reports.tsx & Dashboard.tsx):

- **Slice:** Status (completed/voided), Order Type
- **Dice:** Date Range + Category + Cashier
- **Roll-up:** Hourly → Daily → Weekly → Monthly
- **Drill-down:** Sales KPI → Receipt → Deductions
- **Period-over-Period:** Today vs Yesterday

Owner / Admin View — ADMINONLY

- Financial Reports, Real Net Profit & P&L
- COGS margin breakdown & break-even
- Multi-format Export: PDF & CSV (reportExport)
- Menu recipe builder & Staff user accounts

Store Manager View — MANAGER

- Live Stock Valuation, Low-Stock Warnings
- FEFO Batch Expiry Banners & Spoilage
- Expense management & Cost oversight
- Shift scheduling & Attendance monitoring

Cashier / Staff View — COUNTER

- Fast POS Ordering, Size & Addon picker
- Real-time stock shortage prevention
- Printable & Digital receipt generation
- Daily Sales KPI Cards & Time Clock punch

Data Sources (OLTP Database & Storage) | ETL Transformation & Calculations | Quality Check Decision Diamond | Stop & Alert Routine | Star Schema Warehouse (Facts & Dims) | OLAP Engine & Stakeholder Views | → Directional Data Flow

Text Format Blueprint & Data Flow Connections

INDENTED HIERARCHICAL COMPONENT TREE AND EXPLICIT NODE CONNECTION MATRIX FOR TOOL IMPORT

FIGURE 2.0 ▪ ARCHITECTURE BLUEPRINT

1. LAYER 1: OPERATIONAL DATA SOURCES (OLTP)

- **POS Sales Transactions:** `sales` (id, items JSONB, subtotal, discount, total, payment_method, order_type, cashier_name, status).
- **Inventory & Batches:** `ingredients`, `ingredient_batches` (FEFO expiry, supplier), `stock_movements`, `sale_batch_allocations`.
- **Menu Master Data:** `menu_items`, `menu_item_sizes`, `menu_item_ingredients`, `addons`, `addon_ingredients`, `order_supply_rules`.
- **Expenses:** `expenses` table (id, description, amount, category, date).
- **HR & Attendance:** `employees`, `shift_templates`, `shift_schedules`, `attendance_records`.
- **Offline Cache:** browser `localStorage` fallback (`espro_*`) and Supabase Auth.

2. LAYER 2: ETL PROCESSING PIPELINE (db.ts & orderCalc.ts)

- **Extract:** PostgREST loaders (`loadSales`, `loadIngredients`, etc.) with network error catching & localStorage fallback.
- **Staging Area:** Client in-memory React state + synchronized localStorage cache.
- **Transform Engine (9 Core Steps):**
  - 1. Unit Conversion (`unitConversion.ts`): g, kg, ml, L, oz, shots, pumps, pcs.
  - 2. Recipe Explosion (`computeLine`): size multiplier, addon recipes, `usage_scope`.
  - 3. COGS Snapshot: separate raw ingredient costs from packaging supply costs.
  - 4. Packaging Rules (`computeOrder`): takeout cup, lid, and kraft bag deduction.
  - 5. Net Metric Calculation: `Net Sales = Subtotal (Gross) - Discount`.
  - 6. Missing-Value Imputation: legacy recipe cost fallback; default `dine_in`.
  - 7. FEFO Allocation: earliest expiration date sort; `expire_batches` write-off.
  - 8. Deduplication: deterministic lineKey merging (`item__size__addons__note`).
  - 9. Timezone Normalization: Asia/Manila (UTC+8) day boundaries via `dateUtils.ts`.
- **Quality Check Diamond:** validates stock shortages, unit compatibility, positive amounts, discount ≤ subtotal, PIN code, and completed status.
- **Stop & Alert Routine:** triggers on validation failure → atomic RPC rollback (exception), UI cashier toast, and TopBar alert badge.
- **Load Phase:** atomic PL/pgSQL RPCs (`complete_sale_with_deduction`, `void_sale_with_restoration`, `restock_ingredient`).

3. LAYER 3: DATA WAREHOUSE & STAR SCHEMA (PostgreSQL)

- **Dimensions:** Dim_Date, Dim_Product, Dim_Size, Dim_Addon, Dim_Ingredient_Supply, Dim_Employee_Cashier, Dim_OrderType, Dim_PaymentMethod, Dim_ExpenseCategory, Dim_Batch.
- **Facts:** Fact_Sales (sale line items: qty, gross, discount, net, COGS, profit), Fact_Inventory_Movement, Fact_Expenses, Fact_Attendance.
- **Data Marts:** Sales Mart, Inventory Mart, Finance/Profit Mart (P&L), Workforce Mart.

4. LAYER 4: OLAP ENGINE & ROLE-BASED VIEWS

- **OLAP Logic:** `useMemo` analytical engine (Slice, Dice, Roll-up, Drill-down, Period comparison).
- **Views:** Owner/Admin (P&L, break-even, exports), Store Manager (inventory valuation, batches, staff), Cashier (POS, receipts).

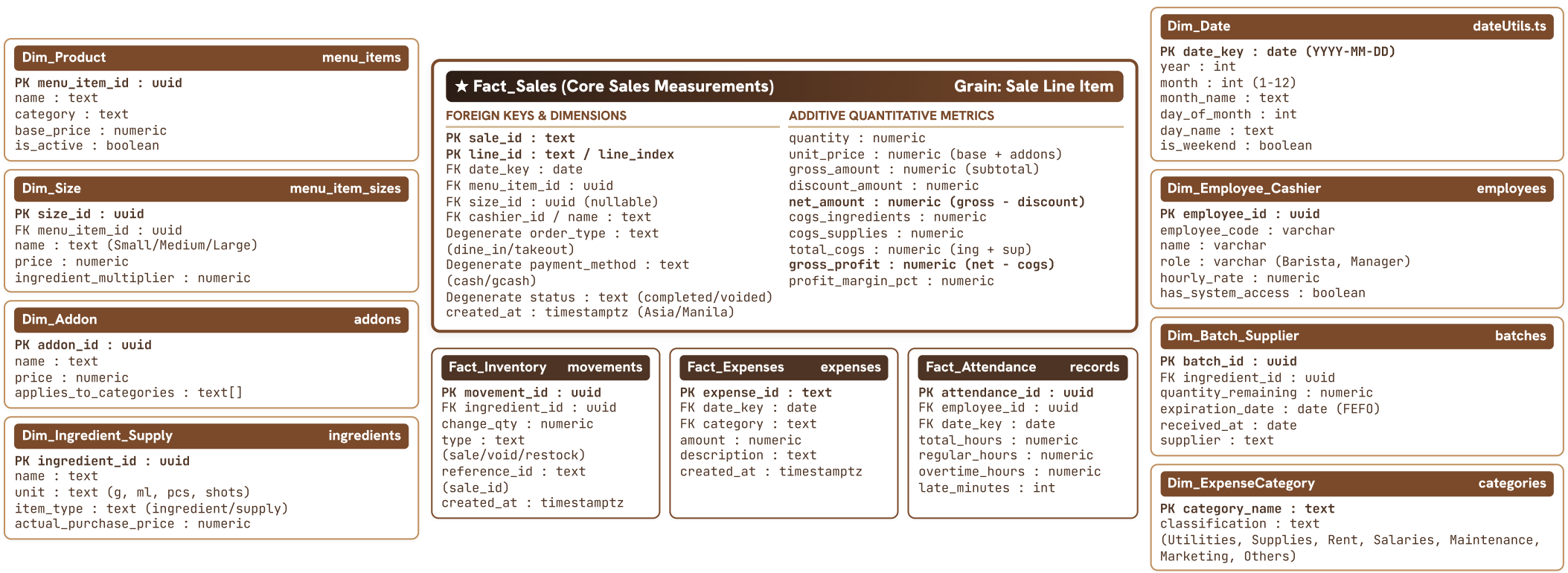
EXPLICIT CONNECTION LIST [SOURCE] → [DESTINATION] (DATA LABEL)

```
[POS Transactions] → [Extract Phase (db.ts)] (Raw Sales JSONB Payload)
[Inventory & Batches] → [Extract Phase (db.ts)] (Stock Levels & FEFO Batches)
[Menu Master & Recipes] → [Extract Phase (db.ts)] (Menu Catalog & Multipliers)
[Expenses Table] → [Extract Phase (db.ts)] (Operating Expense Records)
[HR Time Clock Kiosk] → [Extract Phase (employeeDb.ts)] (Attendance Punch Timestamps)
[Offline localStorage] -.→ [Extract Phase (db.ts)] (Fallback on Network Failure)
[Extract Phase] → [Staging Area Buffer] (In-memory Extracted Datasets)
[Staging Area] → [Transform: Unit Conversion] (Raw Ingredients & Units)
[Unit Conversion] → [Transform: Recipe Explosion] (Normalized Base Units g/ml)
[Recipe Explosion] → [Transform: COGS Snapshot] (Scaled Ingredients & Addons)
[COGS Snapshot] → [Transform: Packaging Rules] (Ingredients vs Supplies COGS)
[Packaging Rules] → [Transform: Net Metric Calc] (Takeout Supplies Appended)
[Net Metric Calc] → [Transform: Missing Fill] (Net = Subtotal - Discount)
[Missing Fill] → [Transform: FEFO Allocation] (Imputed Legacy Cost/OrderType)
[FEFO Allocation] → [Transform: Line Deduplication] (Earliest Expiry Batch List)
[Line Deduplication] → [Transform: Timezone Normal] (Merged lineKey Cart Items)
[Timezone Normal] → [Quality Check Decision Diamond] (Transformed Order Payload)
[Quality Check] → [STOP & ALERT Routine] (On Shortage / Invalid Data FAIL)
[STOP & ALERT Routine] -.→ [TopBar Stock Alert & Cashier Toast] (Visual Warning & Rollback)
[Quality Check] → [Load: Atomic RPCs] (On All Validations PASS)
[Load: Atomic RPCs] → [Fact_Sales & Fact_Inventory] (complete_sale_with_deduction)
[Fact_Sales] → [Sales Mart & Finance Mart] (Aggregated Financial Metrics)
[Fact_Inventory_Movement] → [Inventory Mart] (Stock Valuation & Spoilage)
[Fact_Attendance] → [Workforce Mart] (Labor Hours & Overtime Cost)
[Data Marts] → [OLAP Engine (Reports useMemo)] (Multi-Dimensional Aggregates)
[OLAP Engine] → [Owner, Manager, Cashier Views] (Role-Based Dashboards & Exports)
```

Data Warehouse Multi-Dimensional Star Schema

FACT_SALES CENTRIC STAR SCHEMA WITH CONFORMED DIMENSIONS AND SATELLITE OPERATIONAL FACTS

FIGURE 3.0 ▪ STAR SCHEMA DATA MODEL



ETL Transformation Rules & Data Quality Assurance Matrix

COMPREHENSIVE MAPPING OF SOURCE DATA, BUSINESS RULES, TARGET WAREHOUSE ENTITIES, AND STOP & ALERT HANDLERS

TABLE 1.0 ▪ TRANSFORMATION ENGINE RULES

STEP	STAGE	SOURCE TABLE / CODE	TRANSFORMATION & CALCULATION BUSINESS RULE	TARGET WAREHOUSE ENTITY	ERROR HANDLING & STOP-AND-ALERT
01	EXTRACT	`db.ts` (loadSales, loadIngredients, loadBatches, loadExpenses)	Pulls raw tables from Supabase PostgreSQL via PostgREST. If network failure is detected via `isNetworkError`, gracefully falls back to local cached snapshots.	Client In-Memory Staging State + `localStorage`	Catch network exceptions; notify UI via non-blocking toast; switch data source mode to offline cache.
02	TRANSFORM	`unitConversion.ts` (convertUnitQuantity)	Converts recipe quantities into ingredient inventory units across mass (mg, g, kg, oz, lb), volume (ml, L, fl oz, shots, pumps, cups), and discrete counts (pcs, cans, bags).	`deductionsMap` in inventory units	Throws error on incompatible unit categories (e.g. converting grams to milliliters without specified density).
03	TRANSFORM	`orderCalc.ts` (computeLine)	**Recipe Explosion:** Multiplies raw quantities by `size.ingredient_multiplier`, appends addon recipe ingredients, and applies `usage_scope` ('always' | 'dine_in' | 'takeout').	`CartLine.deductions` & line prices	Missing size or addon recipe row is skipped with zero cost increment; logged to console.
04	TRANSFORM	`orderCalc.ts` (computeLine)	**COGS Snapshot:** Computes cost per serving from `actual_purchase_price` or `cost_per_unit`, split into `cogs_per_serving` vs `cogs_supplies_per_serving`.	`Fact_Sales.cogs_ingredients` & `cogs_supplies`	Fallback to 0 if ingredient cost is unconfigured; flags line item for margin review.
05	TRANSFORM	`orderCalc.ts` (computeOrder)	**Packaging Rules:** Evaluates active `order_supply_rules`. Injects takeout cups, lids, and kraft bags into order deductions if `order_type == 'takeout'`.	`orderDeductions` & `orderSuppliesCogs`	Supplies shortage treated under overall order shortage check before sale confirmation.
06	TRANSFORM	`POS.tsx` & `orderCalc.ts`	**Net Amount Calculation:** Computes financial metric strictly as: `Net Sales = Subtotal (Gross) - Discount Amount`. Computes `Gross Profit = Net Sales - Total COGS`.	`Fact_Sales.net_amount` & `profit_amount`	Discount clamped to subtotal (discount cannot exceed gross); total cannot be negative.
07	TRANSFORM	`db.ts` (loadBatches)	**FEFO Allocation:** Sorts batches by earliest expiration date first (nulls last), then earliest `received_at`. Depleted batches marked inactive; expired batches written off.	`ingredient_batches` & `sale_batch_allocations`	Expired batches blocked from sale deduction; triggers automated spoilage write-off prompt.
08	TRANSFORM	`orderCalc.ts` (lineKey)	**Cart Deduplication:** Generates deterministic hash: `item_id__size_id__sortedAddons__note`. Merges identical line configurations to avoid duplicate line records.	Consolidated Cart Line Item	Case-insensitive note trimming prevents accidental split items on counter entry.
09	TRANSFORM	`dateUtils.ts` (startOfDayLocal)	**Timezone Normalization:** Anchors timestamps to local Asia/Manila (UTC+8) time without day-shifting bugs, ensuring accurate daily financial reconciliation.	`Dim_Date.date_key` & Fact foreign keys	Date strings lacking timezone parsed as local noon (12:00:00) to avoid calendar shift.
10	QUALITY CHECK	`orderCalc.ts` (shortages)	**Stock Shortage Verification:** Evaluates combined ingredient needs against available `stock_quantity` across all active lines.	Quality Gate Decision Diamond	**STOP & ALERT:** Sale blocked; displays shortage item modal with required vs available quantities.
11	LOAD (ATOMIC)	Supabase RPC (complete_sale_with_deduction)	**Atomic Database Load:** Single PostgreSQL PL/pgSQL transaction inserts sale record, deducts batch & ingredient inventory, and writes stock_movements ledger.	`Fact_Sales` & `Fact_Inventory_Movement`	**STOP & ALERT:** Any integrity or shortage error raises exception → transaction rolls back 100% atomically.

System-to-Data-Warehouse Implementation Mapping Matrix

ACADEMIC GROUNDING: ENTERPRISE BUSINESS INTELLIGENCE CONCEPTS MAPPED TO REAL ESPRO POS SOURCE CODE

TABLE 2.0 ▪ ARCHITECTURE MAPPING & PROOF

DATA WAREHOUSE CONCEPT	ESPRO POS SYSTEM IMPLEMENTATION	CODE EVIDENCE & ARTIFACTS	ARCHITECTURAL CLASSIFICATION
Operational Data Store (ODS)	Transactional Supabase PostgreSQL tables storing active counter transactions, raw stock items, employee profiles, and operational store expenses.	`supabase_schema.sql`, `supabase_recipe_schema.sql`, `supabase_migration_employees.sql`	Normalized relational schema (3NF) optimized for high-speed counter POS read/write throughput.
Staging Area	Client-side in-memory React state buffers and synchronized browser `localStorage` cache storing raw, uncommitted batches, cart drafts, and operational pulls.	`src/lib/db.ts` (getLocalBatches, getLocalIngredients), `src/pages/POS.tsx`	Temporary decoupled staging preventing lock contention on operational tables.
ETL Processing Engine	TypeScript algorithmic pipeline performing unit conversions, multi-tier recipe explosions, COGS snapshots, and net sales calculations.	`src/lib/orderCalc.ts` (computeLine, computeOrder), `src/lib/unitConversion.ts`	Distributed Client-Side Execution Layer with backend PL/pgSQL atomic RPC execution.
Data Quality & Anomaly Handling	Pre-flight shortage validation, unit category validation, and atomic PL/pgSQL rollback triggers stopping processing on anomalies with cashier notifications.	`orderCalc.ts` (shortages check), `complete_sale_with_deduction` (RAISE EXCEPTION)	**"Stop and Alert Team"** Routine implemented via atomic rollback and toast UI banners.
Conformed Dimensions	Standardized context entities shared across facts: Dim_Date, Dim_Product, Dim_Size, Dim_Addon, Dim_Ingredient, Dim_Employee_Cashier, Dim_OrderType.	`menu_items`, `menu_item_sizes`, `ingredients`, `employees`, `dateUtils.ts`	Conformed analytical dimensions enabling cross-functional slice & dice across sales and inventory.
Core Fact Tables	Fact_Sales storing granular line item measurements (Quantity, Gross, Discount, Net Sales, Ingredient COGS, Supplies COGS, Profit Amount).	`sales` (items JSONB + financial totals), `stock_movements`, `expenses`	Transaction Grain & Periodic Snapshot Fact Tables storing additive quantitative metrics.
Localized Data Marts	Specialized analytical aggregations: Sales Mart (revenue trends), Inventory Mart (valuation & turnover), Finance Mart (P&L statement), Workforce Mart (labor hours).	`src/pages/Reports.tsx` (metrics useMemo), `src/pages/Dashboard.tsx`, `src/pages/Inventory.tsx`	Domain-specific analytical marts providing fast query response for targeted operational roles.
OLAP Processing Logic	In-memory multi-dimensional analytical engine supporting slice, dice, roll-up (hourly to monthly), drill-down (KPI to line deductions), and period comparison.	`Reports.tsx` (salesFiltered useMemo, datePresets), `dateUtils.ts` (getPresetDateRange)	Real-Time Desktop & Web OLAP Engine with reactive sub-second re-aggregation upon filter changes.
Role-Based Analytics & BI	Role-gated dashboards serving Executive Owner (P&L, break-even, PDF/CSV export), Store Manager (inventory, batches, staff), and Cashier (counter KPIs).	`src/App.tsx` (ProtectedRoute adminOnly), `src/lib/reportExport.ts` (exportFullReportPdf)	Stakeholder Role-Based Access Control (RBAC) delivering tailored business intelligence.

Academic Architecture Note & Honest Design Disclosure:
In Espro POS, the Data Warehouse layer is an *in-database logical schema and client-side OLAP engine* implemented within Supabase PostgreSQL, atomic PL/pgSQL stored procedures, and React 19 memoized pipelines—rather than an over-engineered detached MPP cloud warehouse (such as Snowflake or Google BigQuery). This architectural design is deliberate and optimal for small-to-medium specialty coffee retail: it eliminates external ETL pipeline latency, guarantees atomic transactional consistency during peak counter ordering, provides instant offline operational resilience via `localStorage`, and avoids the significant cloud compute costs of enterprise data warehouses while achieving full dimensional reporting fidelity.